

Cricket Player Trading Platform (LineUp) - Product Requirements Document (PRD)

Table of Contents

1. Overview
2. Tech Stack
3. Microservices Architecture
4. Database Schema (Core Tables)
5. Service Details
6. API Gateway
7. Kubernetes Deployment
8. Security & Compliance
9. MVP Scope (Phase 1)
10. Future Enhancements (Phase 2+)
11. Appendix: Key Flows
12. User Journeys & Edge Cases
13. Mobile & Web Integration
14. Real-Time Data Flows
15. Monitoring & Observability
16. DevOps & CI/CD
17. Migration & Extensibility
18. Glossary
19. References

1. Overview

A real-time fantasy stock market for cricket fans, enabling users to buy/sell player stocks based on live match performance. Built for scale, speed, and reliability using a microservices architecture.

1.1. Vision

To revolutionize cricket fan engagement by merging the excitement of live matches with the thrill of real-time trading, creating a dynamic, skill-based platform where users can profit from their cricketing insights.

1.2. Mission

Deliver a secure, scalable, and intuitive platform that empowers users to trade cricket player stocks, leveraging live data, advanced analytics, and a robust risk management system.

1.3. Goals

- Achieve sub-5s price refresh for all live trades
- Support 1M+ concurrent users during major tournaments
- Ensure regulatory compliance (KYC/AML, GDPR)
- Provide seamless experience across mobile and web

1.4. Stakeholders

- Product Owners
- Engineering Team (Backend, Frontend, DevOps)
- QA & Security
- Legal & Compliance
- End Users (Traders, Fans)
- Admins & Moderators

2. Tech Stack (Expanded)

Overview

- The CricketIQ platform leverages a modern, modular tech stack designed for scalability, real-time performance, developer productivity, and cross-platform reach.
- The stack is selected to support microservices, high-frequency trading, live data feeds, and seamless user experiences on both web and mobile.

Backend

- **Language:** Go (Golang)
 - Chosen for its concurrency model (goroutines, channels), strong typing, and performance.
 - Ideal for real-time trading, microservices, and low-latency APIs.

- **Frameworks:** Standard library, Gin, gRPC
 - Gin for REST APIs, gRPC for internal service communication.
- **Microservices:** Each core domain (users, trading, pricing, portfolio, match data, notifications, admin, community) is a separate Go service.
- **API Gateway:** NGINX or Envoy for routing, load balancing, and security.
- **Authentication:** JWT for stateless auth, OAuth2 for social logins, 2FA for enhanced security.
- **Background Jobs:** Go workers for async tasks (settlement, notifications, data sync).

Frontend

- **Web:** Next.js (React)
 - Server-side rendering (SSR) for SEO, static site generation (SSG) for speed.
 - WebSocket integration for real-time price updates, order book, and notifications.
 - TailwindCSS for rapid, consistent UI development.
- **Mobile:** React Native
 - Shared codebase for iOS and Android, native performance.
 - Deep linking, push notifications, and offline support.
 - Integration with REST and WebSocket APIs for live trading and alerts.

Real-Time Engine

- **WebSocket:** For live price feeds, order book updates, and instant notifications.
- **Redis Pub/Sub:** Low-latency event distribution between services (e.g., price changes, trade execution).
- **Kafka:** High-throughput, persistent event streaming for audit logs, analytics, and cross-service workflows.
- **Socket.IO:** Optional for mobile/web fallback or advanced real-time features.

Database Layer

- **PostgreSQL:**
 - ACID-compliant, relational database for transactional data (users, orders, trades, portfolios, wallets).
 - Partitioning and indexing for high performance at scale.
- **MongoDB:**
 - NoSQL, document-oriented database for semi-structured data (ball-by-ball stats, logs, analytics).
 - Flexible schema for evolving data models and high write throughput.

- **Redis:**

- In-memory cache for hot data (order books, price snapshots, session tokens).
- Used for rate limiting, leaderboards, and temporary state.

DevOps & Deployment

- **Kubernetes (K8s):**

- Container orchestration for all microservices, supporting auto-scaling, rolling updates, and self-healing.

- Helm charts for repeatable, versioned deployments.

- **CI/CD:**

- GitHub Actions or Jenkins for automated build, test, and deployment pipelines.
- Docker for containerization of all services and frontends.

- **Secrets Management:**

- HashiCorp Vault or K8s Secrets for API keys, DB credentials, and sensitive configs.

- **Monitoring & Logging:**

- Prometheus for metrics, Grafana for dashboards.
- ELK Stack (Elasticsearch, Logstash, Kibana) for centralized logging and search.
- Sentry for error tracking and alerting.

Security

- **Encryption:** SSL/TLS for all network traffic, AES-256 for data at rest.

- **Access Control:** RBAC for admin tools, scoped API tokens for service-to-service auth.

- **Compliance:** GDPR/CCPA support, audit logs, and regular security reviews.

- **DDoS Protection:** Cloudflare or AWS Shield for edge security.

Third-Party Integrations

- **Sports Data APIs:** SportRadar, CricAPI, Roanuz for live match stats, player data, and schedules.

- **Payment Providers:** Stripe, Razorpay, or similar for fiat on/off ramps.

- **Notification Providers:** Twilio (SMS), Firebase (push), SendGrid (email).

- **Analytics:** Google Analytics, Mixpanel, or Segment for user behavior tracking.

Testing & Quality Assurance

- **Unit & Integration Tests:** Go test, Jest (JS), Detox (mobile).

- **Load Testing:** Locust, k6, or JMeter for simulating high-traffic scenarios.

- **Staging Environments:** Full-stack replicas for pre-production validation.

Developer Experience

- **Monorepo or Polyrepo:** Managed via Nx, Lerna, or native Git for code sharing and modularity.
- **API Documentation:** OpenAPI/Swagger for REST, Protobuf for gRPC.
- **Code Quality:** Prettier, ESLint, GolangCI-Lint, Husky for pre-commit hooks.
- **Onboarding:** Automated scripts, docs, and sample data for rapid developer ramp-up.

Scalability & Extensibility

- **Horizontal Scaling:** K8s auto-scaling for stateless services, DB read replicas for heavy queries.
- **Service Mesh:** Istio or Linkerd for advanced routing, observability, and security.
- **Feature Flags:** LaunchDarkly or custom solution for safe, gradual rollouts.
- **Plugin Architecture:** Support for custom modules (e.g., new sports, analytics, or trading strategies).

Example Architecture Diagram

...

[Web/Mobile Clients]



[API Gateway] <-> [WebSocket Server]



[Microservices: User, Trading, Pricing, Portfolio, Match Data, Notification, Admin, Community]



[PostgreSQL] [MongoDB] [Redis] [Kafka]



[Monitoring, Logging, Analytics]

...

Future-Proofing

- Modular, API-first design enables rapid integration of new features, partners, or asset classes.
- Cloud-agnostic deployment supports AWS, GCP, Azure, or hybrid models.
- Continuous evaluation of emerging tech (e.g., WebAssembly, serverless, AI/ML) for future enhancements.

``````markdown

## ## 3. Microservices Architecture

### ### 3.1. Services

- **User Service:** Registration, KYC, profile, authentication, tiering
- **Trading Service:** Order matching, trade execution, order book, position management
- **Pricing Engine:** Real-time price calculation, volatility, risk controls
- **Portfolio Service:** User holdings, PnL, transaction history
- **Match Data Service:** Live match feed, player stats, schedule
- **Notification Service:** Push, in-app, email, SMS alerts
- **Admin Service:** User management, fraud detection, analytics
- **Community Service:** Leaderboards, chat, achievements

#### #### 3.1.1. User Service

- Handles user registration, authentication (JWT, OAuth), KYC document upload, user profile management, tiering, and referral logic.
- Integrates with Notification Service for alerts and with Admin Service for moderation.
- Exposes REST/gRPC endpoints for all user operations.

#### #### 3.1.2. Trading Service

- Manages order placement, cancellation, order book, and trade execution.
- Implements FIFO/priority queue matching logic.
- Handles margin checks, position lifecycle, and trade history.
- Publishes trade events to Kafka/Redis for downstream services.

#### #### 3.1.3. Pricing Engine

- Consumes live match data and trade events to recalculate player prices in real time.
- Supports multiple pricing models (linear, exponential, regression, volatility-based).
- Applies circuit breakers, cooldowns, and risk controls.
- Publishes price updates to WebSocket and event bus.

#### #### 3.1.4. Portfolio Service

- Tracks user holdings, open/closed positions, PnL, and margin status.
- Provides APIs for portfolio dashboard, historical trades, and ROI analytics.
- Integrates with Wallet Service for balance and margin management.

#### #### 3.1.5. Match Data Service

- Ingests live match data from third-party APIs (SportRadar, CricAPI, Roanuz).
- Normalizes and stores player stats, match status, and ball-by-ball events.
- Exposes APIs for schedule, live scores, and player performance.

#### #### 3.1.6. Notification Service

- Manages user notification preferences and delivery (push, email, SMS, in-app).
- Triggers alerts for price changes, match events, PnL/risk updates.
- Integrates with external providers (Twilio, Firebase, SendGrid).

#### #### 3.1.7. Admin Service

- Provides dashboards for user/KYC management, trade monitoring, fraud detection, and analytics.
- Manages content (banners, promos, announcements).
- Exposes secure admin APIs with RBAC.

#### #### 3.1.8. Community Service

- Handles leaderboards, achievements, badges, chat/forum, influencer portfolios, and copy trades.
- Integrates with Trading and Portfolio services for real-time updates.

---

### ### 3.2. Communication

- **API Gateway:** REST/GraphQL for client access
- **Internal:** gRPC/REST between services
- **Real-time:** WebSocket (for price feed, order book)
- **Event Bus:** Kafka/Redis for pub-sub events (trade executed, price update, match event)

---

## 4. Database Schema (Core Tables)

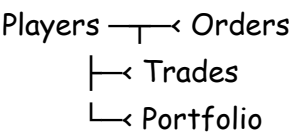
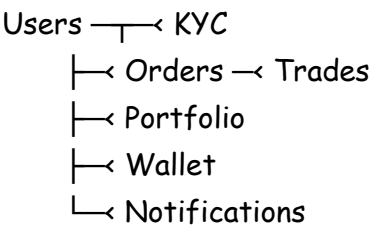
---

### 4.0. Schema Overview

The platform uses a hybrid database approach, leveraging PostgreSQL for transactional, relational data and MongoDB for flexible, high-volume, semi-structured data (such as ball-by-ball stats and logs). All schemas are designed for scalability, auditability, and regulatory compliance.

#### Entity Relationship Diagram (ERD)

...



PlayerStats (MongoDB): Linked by player\_id, match\_id

...

---

#### 4.1. Users

| Field         | Type    | Description               |
|---------------|---------|---------------------------|
| id            | UUID    | Primary Key               |
| email         | VARCHAR | Unique, login             |
| phone         | VARCHAR | Optional, login           |
| password_hash | VARCHAR | Hashed password           |
| kyc_status    | ENUM    | pending/verified/rejected |
| name          | VARCHAR |                           |



|                 |           |                 |  |
|-----------------|-----------|-----------------|--|
| username        | VARCHAR   | Unique          |  |
| avatar_url      | VARCHAR   | Profile picture |  |
| favorite_team   | VARCHAR   |                 |  |
| favorite_player | VARCHAR   |                 |  |
| country         | VARCHAR   |                 |  |
| timezone        | VARCHAR   |                 |  |
| tier            | ENUM      | basic/premium   |  |
| referral_code   | VARCHAR   |                 |  |
| created_at      | TIMESTAMP |                 |  |
| updated_at      | TIMESTAMP |                 |  |

#### ##### Field Explanations

- **\*\*id:\*\*** Universally unique identifier for each user (UUID v4).
- **\*\*email/phone:\*\*** Used for login and notifications; must be unique.
- **\*\*password\_hash:\*\*** Securely hashed using bcrypt/argon2.
- **\*\*kyc\_status:\*\*** Enum (pending, verified, rejected); controls access to withdrawals and trading.
- **\*\*tier:\*\*** User tier (basic/premium) for feature gating and rewards.
- **\*\*referral\_code:\*\*** Used for referral tracking and rewards.
- **\*\*created\_at/updated\_at:\*\*** Timestamps for auditing and analytics.

#### ##### Relationships

- One-to-many with KYC, Orders, Portfolio, Wallet, Notifications.

#### ##### Additional Notes

- Email and phone are validated at registration.
- GDPR compliance: Users can request data export or deletion.

#### ##### Sample Query

```
```sql
SELECT * FROM users WHERE email = 'user@example.com';
```
```

#### ##### Create Table

```
```sql
CREATE TABLE users (
```

```

id UUID PRIMARY KEY,
email VARCHAR(255) UNIQUE NOT NULL,
phone VARCHAR(20) UNIQUE,
password_hash VARCHAR(255) NOT NULL,
kyc_status VARCHAR(16) NOT NULL,
name VARCHAR(100),
username VARCHAR(50) UNIQUE NOT NULL,
avatar_url VARCHAR(255),
favorite_team VARCHAR(50),
favorite_player VARCHAR(50),
country VARCHAR(50),
timezone VARCHAR(50),
tier VARCHAR(16) NOT NULL,
referral_code VARCHAR(32),
created_at TIMESTAMP NOT NULL DEFAULT NOW(),
updated_at TIMESTAMP NOT NULL DEFAULT NOW()
);

```

4.2. KYC

| Field | Type | Description |
|--------------|-----------|-------------------|
| id | UUID | Primary Key |
| user_id | UUID | FK to users |
| doc_type | VARCHAR | Govt ID, etc. |
| doc_number | VARCHAR | |
| doc_url | VARCHAR | File storage link |
| status | ENUM | pending/verified |
| submitted_at | TIMESTAMP | |

Field Explanations

- **user_id:** Foreign key to Users; enforces one KYC per user.
- **doc_type/doc_number/doc_url:** Stores document metadata and secure file link.
- **status:** Enum (pending, verified, rejected); only verified users can withdraw.

Relationships

- Many-to-one with Users.

Additional Notes

- KYC docs are encrypted and access-logged.

Sample Query

```
```sql
SELECT * FROM kyc WHERE user_id = '...';
```
```

Create Table

```
```sql
CREATE TABLE kyc (
 id UUID PRIMARY KEY,
 user_id UUID REFERENCES users(id),
 doc_type VARCHAR(32) NOT NULL,
 doc_number VARCHAR(64),
 doc_url VARCHAR(255),
 status VARCHAR(16) NOT NULL,
 submitted_at TIMESTAMP NOT NULL DEFAULT NOW()
);
```
```

4.3. Players

| Field | Type | Description |
|-----------|---------|-------------------|
| id | UUID | Primary Key |
| name | VARCHAR | |
| team | VARCHAR | |
| role | VARCHAR | Batter/Bowler/All |
| is_active | BOOLEAN | |

Field Explanations

- ****id:**** Unique player identifier, mapped to external cricket APIs.
- ****team:**** Team name (e.g., India, Australia).
- ****role:**** Enum (Batter, Bowler, All-rounder, Wicketkeeper).

- **is_active:** Indicates if player is currently active in the system.

Relationships

- One-to-many with Orders, Trades, Portfolio.

Additional Notes

- Player data is periodically synced with official cricket data providers.

Sample Query

```
```sql
SELECT * FROM players WHERE team = 'India';
```
```

Create Table

```
```sql
CREATE TABLE players (
 id UUID PRIMARY KEY,
 name VARCHAR(100) NOT NULL,
 team VARCHAR(50) NOT NULL,
 role VARCHAR(32) NOT NULL,
 is_active BOOLEAN NOT NULL DEFAULT TRUE
);
```
```

4.4. Matches

| Field | Type | Description |
|------------|-----------|----------------------|
| id | UUID | Primary Key |
| series | VARCHAR | IPL, WC, etc. |
| start_time | TIMESTAMP | |
| end_time | TIMESTAMP | |
| status | ENUM | scheduled/live/ended |

Field Explanations

- **series:** Tournament or series name (e.g., IPL 2026).

- **status:** Enum (scheduled, live, ended, abandoned).

Relationships

- One-to-many with Orders, Trades, Portfolio.

Additional Notes

- Match status is updated in real time for trading logic.

Sample Query

```
```sql
```

```
SELECT * FROM matches WHERE series = 'IPL 2026' AND status = 'live';
```

```
```
```

Create Table

```
```sql
```

```
CREATE TABLE matches (
 id UUID PRIMARY KEY,
 series VARCHAR(100) NOT NULL,
 start_time TIMESTAMP NOT NULL,
 end_time TIMESTAMP,
 status VARCHAR(16) NOT NULL
```

```
);
```

```
```
```

4.5. PlayerStats (MongoDB)

- match_id
- player_id
- runs, wickets, catches, SR, economy, milestones, etc.
- ball_by_ball: [{over, ball, runs, wickets, event}]

Field Explanations

- ****match_id/player_id:**** Composite key for fast lookups.
- ****runs, wickets, catches, SR, economy, milestones:**** All key performance metrics.
- ****ball_by_ball:**** Array of objects, each representing a ball event (over, ball, runs, wickets, event type).

Relationships

- Linked to Players and Matches by IDs.

Additional Notes

- Used for analytics, pricing, and user-facing stats overlays.

Sample Query (MongoDB)

```
```js
db.playerstats.find({ match_id: '...', player_id: '...' })
```
```

Create Collection (MongoDB)

```
```js
db.createCollection('playerstats')
// Documents: {
// match_id: UUID,
// player_id: UUID,
// runs: Number,
// wickets: Number,
// catches: Number,
// sr: Number,
// economy: Number,
// milestones: [String],
// ball_by_ball: [{ over: Number, ball: Number, runs: Number, wickets: Number, event:
String }]
// }
```
```

4.6. Orders

| Field | Type | Description |
|------------|------|----------------|
| id | UUID | Primary Key |
| user_id | UUID | FK to users |
| player_id | UUID | FK to players |
| match_id | UUID | FK to matches |
| type | ENUM | buy/sell/short |
| order_type | ENUM | market/limit |

| | | |
|------------|-----------|--------------------|
| price | NUMERIC | |
| quantity | INTEGER | |
| status | ENUM | open/filled/cancel |
| created_at | TIMESTAMP | |

Field Explanations

- ****type:**** Enum (buy, sell, short); determines order logic.
- ****order_type:**** Enum (market, limit); market executes instantly, limit waits for price.
- ****status:**** Enum (open, filled, cancelled, expired).

Relationships

- Many-to-one with Users, Players, Matches.
- One-to-many with Trades.

Additional Notes

- Orders are soft-deleted for audit and compliance.

Sample Query

```
```sql
SELECT * FROM orders WHERE player_id = '...' AND match_id = '...' AND status = 'open';
```
```

Create Table

```
```sql
CREATE TABLE orders (
 id UUID PRIMARY KEY,
 user_id UUID REFERENCES users(id),
 player_id UUID REFERENCES players(id),
 match_id UUID REFERENCES matches(id),
 type VARCHAR(16) NOT NULL,
 order_type VARCHAR(16) NOT NULL,
 price NUMERIC(12,2) NOT NULL,
 quantity INTEGER NOT NULL,
 status VARCHAR(16) NOT NULL,
 created_at TIMESTAMP NOT NULL DEFAULT NOW()
);
```

...

#### #### 4.7. Trades

| Field         | Type      | Description |  |
|---------------|-----------|-------------|--|
| id            | UUID      | Primary Key |  |
| buy_order_id  | UUID      |             |  |
| sell_order_id | UUID      |             |  |
| player_id     | UUID      |             |  |
| match_id      | UUID      |             |  |
| price         | NUMERIC   |             |  |
| quantity      | INTEGER   |             |  |
| executed_at   | TIMESTAMP |             |  |

#### ##### Field Explanations

- **buy\_order\_id/sell\_order\_id**: Links to Orders for full trade traceability.
- **executed\_at**: Timestamp for trade settlement and audit.

#### ##### Relationships

- Many-to-one with Orders, Players, Matches.

#### ##### Additional Notes

- Immutable for regulatory compliance.

#### ##### Sample Query

```
```sql
SELECT * FROM trades WHERE player_id = '...' AND match_id = '...';
```
```

#### ##### Create Table

```
```sql
CREATE TABLE trades (
  id UUID PRIMARY KEY,
  buy_order_id UUID REFERENCES orders(id),
  sell_order_id UUID REFERENCES orders(id),
  player_id UUID REFERENCES players(id),
```



```

    match_id UUID REFERENCES matches(id),
    price NUMERIC(12,2) NOT NULL,
    quantity INTEGER NOT NULL,
    executed_at TIMESTAMP NOT NULL DEFAULT NOW()
);
...

```

4.8. Portfolio

| Field | Type | Description | |
|---------------|---------|-------------|-------|
| ----- | ----- | ----- | ----- |
| id | UUID | Primary Key | |
| user_id | UUID | | |
| player_id | UUID | | |
| match_id | UUID | | |
| position_type | ENUM | long/short | |
| quantity | INTEGER | | |
| avg_price | NUMERIC | | |
| pnl | NUMERIC | | |
| status | ENUM | open/closed | |

Field Explanations

- **position_type:** Enum (long, short); tracks direction of trade.
- **avg_price:** Weighted average price for open positions.
- **pnl:** Real-time profit/loss, updated by Pricing Engine.

Relationships

- Many-to-one with Users, Players, Matches.

Additional Notes

- Used for dashboard, risk, and margin calculations.

Sample Query

```

```sql
SELECT * FROM portfolio WHERE user_id = '...' AND status = 'open';
...

```

#### ##### Create Table

```sql

```
CREATE TABLE portfolio (  
  id UUID PRIMARY KEY,  
  user_id UUID REFERENCES users(id),  
  player_id UUID REFERENCES players(id),  
  match_id UUID REFERENCES matches(id),  
  position_type VARCHAR(16) NOT NULL,  
  quantity INTEGER NOT NULL,  
  avg_price NUMERIC(12,2) NOT NULL,  
  pnl NUMERIC(12,2) NOT NULL,  
  status VARCHAR(16) NOT NULL
```

```
);
```

```

#### ### 4.9. Wallet

| Field   | Type    | Description     |
|---------|---------|-----------------|
| id      | UUID    | Primary Key     |
| user_id | UUID    |                 |
| balance | NUMERIC |                 |
| margin  | NUMERIC | For shorts      |
| locked  | NUMERIC | For open orders |

#### ##### Field Explanations

- **balance:** User's available funds for trading.
- **margin:** Funds locked for short trades.
- **locked:** Funds reserved for open orders.

#### ##### Relationships

- One-to-one with Users.

#### ##### Additional Notes

- All wallet changes are auditable and reversible.

#### ##### Sample Query

```
```sql
SELECT * FROM wallet WHERE user_id = '...';
```
```

#### ##### Create Table

```
```sql
CREATE TABLE wallet (
  id UUID PRIMARY KEY,
  user_id UUID REFERENCES users(id),
  balance NUMERIC(12,2) NOT NULL,
  margin NUMERIC(12,2) NOT NULL,
  locked NUMERIC(12,2) NOT NULL
);
```
```

#### ### 4.10. Notifications

| Field      | Type      | Description      |
|------------|-----------|------------------|
| id         | UUID      | Primary Key      |
| user_id    | UUID      |                  |
| type       | ENUM      | price/match/risk |
| message    | TEXT      |                  |
| read       | BOOLEAN   |                  |
| created_at | TIMESTAMP |                  |

#### ##### Field Explanations

- **type:** Enum (price, match, risk, marketing, system).
- **message:** Localized, templated message body.
- **read:** Boolean; unread notifications trigger badge/alert.

#### ##### Relationships

- Many-to-one with Users.

#### ##### Additional Notes

- Notifications are purged after 30 days for storage efficiency.

#### ##### Sample Query

```
```sql
SELECT * FROM notifications WHERE user_id = '...' AND read = false;
```
```

#### ##### Create Table

```
```sql
CREATE TABLE notifications (
  id UUID PRIMARY KEY,
  user_id UUID REFERENCES users(id),
  type VARCHAR(32) NOT NULL,
  message TEXT NOT NULL,
  read BOOLEAN NOT NULL DEFAULT FALSE,
  created_at TIMESTAMP NOT NULL DEFAULT NOW()
);
```

```

### ## 5. Service Details (Expanded)

#### ### Overview

- The CricketIQ platform is built on a microservices architecture, with each core domain implemented as a decoupled, independently deployable *Go* service.
- Services communicate via REST/gRPC APIs, event buses (Kafka/Redis), and are orchestrated by Kubernetes for scalability and resilience.

#### ### 5.1. User Service

- Handles user registration, authentication (JWT, OAuth), KYC document upload, profile management, tiering, and referrals.
  - Integrates with Notification Service for alerts and Admin Service for moderation.
  - Exposes REST/gRPC endpoints for all user operations.
  - Implements password hashing, email/phone verification, and 2FA.
  - KYC workflow includes document upload, status tracking, and admin review.
  - User tiering logic for feature gating and rewards.
  - File Structure Example:
- ```
```
```

```
user-service/
 main.go
 api/
 handlers.go
 routes.go
 models/
 user.go
 kyc.go
 db/
 postgres.go
 middleware/
 auth.go
 validation.go
 service/
 user.go
 kyc.go
 config/
 config.go
 proto/
 user.proto
 Dockerfile
 go.mod
 README.md
```
```

5.2. Trading Service

- Manages order placement, cancellation, order book, and trade execution.
- Implements FIFO/priority queue matching logic and margin checks.
- Publishes trade events to Kafka/Redis for downstream services.
- Supports advanced order types (limit, stop-loss, take-profit).
- File Structure Example:

```
```
```

```
trading-service/
 main.go
 api/
 handlers.go
```

```
 routes.go
models/
 order.go
 trade.go
db/
 postgres.go
 redis.go
service/
 orderbook.go
 matching.go
events/
 publisher.go
 consumer.go
config/
 config.go
proto/
 trading.proto
Dockerfile
go.mod
README.md
` ``
```

### ### 5.3. Pricing Engine

- Consumes live match data and trade events to recalculate player prices in real time.
  - Supports multiple pricing models (linear, exponential, regression, volatility-based).
  - Applies circuit breakers, cooldowns, and risk controls.
  - Publishes price updates to WebSocket and event bus.
  - File Structure Example:
- ```
` ``
```

```
pricing-engine/
    main.go
    api/
        handlers.go
        routes.go
models/
    price.go
```

```
performance.go
service/
  pricing.go
  risk.go
events/
  publisher.go
  consumer.go
config/
  config.go
proto/
  pricing.proto
Dockerfile
go.mod
README.md
` ``
```

5.4. Portfolio Service

- Tracks user holdings, open/closed positions, PnL, and margin status.
 - Provides APIs for portfolio dashboard, historical trades, and ROI analytics.
 - Integrates with Wallet Service for balance and margin management.
 - File Structure Example:
- ```
` ``
```

```
portfolio-service/
 main.go
 api/
 handlers.go
 routes.go
 models/
 portfolio.go
 db/
 postgres.go
 service/
 portfolio.go
 analytics.go
 config/
 config.go
```

```
proto/
 portfolio.proto
Dockerfile
go.mod
README.md
...
```

### ### 5.5. Match Data Service

- Ingests live match data from third-party APIs (SportRadar, CricAPI, Roanuz).
- Normalizes and stores player stats, match status, and ball-by-ball events.
- Exposes APIs for schedule, live scores, and player performance.
- File Structure Example:  
...

```
matchdata-service/
 main.go
 api/
 handlers.go
 routes.go
 models/
 match.go
 playerstats.go
 db/
 postgres.go
 mongo.go
 service/
 ingest.go
 normalize.go
 events/
 publisher.go
 consumer.go
 config/
 config.go
 proto/
 matchdata.proto
Dockerfile
go.mod
```



README.md

...

### ### 5.6. Notification Service

- Manages user notification preferences and delivery (push, email, SMS, in-app).
- Triggers alerts for price changes, match events, PnL/risk updates.
- Integrates with external providers (Twilio, Firebase, SendGrid).
- File Structure Example:

...

notification-service/

main.go

api/

handlers.go

routes.go

models/

notification.go

db/

postgres.go

service/

notification.go

delivery.go

providers/

twilio.go

firebase.go

sendgrid.go

config/

config.go

proto/

notification.proto

Dockerfile

go.mod

README.md

...

### ### 5.7. Admin Service

- Provides dashboards for user/KYC management, trade monitoring, fraud detection, and analytics.

- Manages content (banners, promos, announcements).

- Exposes secure admin APIs with RBAC.

- File Structure Example:

...

admin-service/

main.go

api/

handlers.go

routes.go

models/

admin.go

audit.go

db/

postgres.go

service/

admin.go

moderation.go

config/

config.go

proto/

admin.proto

Dockerfile

go.mod

README.md

...

#### ### 5.8. Community Service

- Handles leaderboards, achievements, badges, chat/forum, influencer portfolios, and copy trades.

- Integrates with Trading and Portfolio services for real-time updates.

- File Structure Example:

...

community-service/

main.go

```
api/
 handlers.go
 routes.go
models/
 leaderboard.go
 chat.go
 achievement.go
db/
 postgres.go
 redis.go
service/
 community.go
 chat.go
config/
 config.go
proto/
 community.proto
Dockerfile
go.mod
README.md
...

```

### ## 6. API Gateway

- REST/GraphQL endpoints for mobile/web
- Auth, rate limiting, request validation
- WebSocket endpoints for real-time feeds

---

### ## 7. Kubernetes Deployment

- Each microservice as a separate container
- Helm charts for deployment, scaling, config
- Horizontal Pod Autoscaling (HPA) for real-time services
- Secrets management (K8s secrets, Vault)
- CI/CD: GitHub Actions/Jenkins → Docker → K8s

- Monitoring: Prometheus, Grafana, ELK

---

### ## 8. Security & Compliance

- JWT auth, 2FA, RBAC
- KYC/AML, age-gate (18+)
- SSL everywhere, DB encryption
- GDPR compliance, audit logs

---

### ## 9. MVP Scope (Phase 1)

- User Management (KYC-lite)
- Player Marketplace (Day Trading only)
- Portfolio Dashboard
- Live Match Integration
- Basic Dynamic Pricing Engine
- OMS (Buy/Sell/Short)
- Risk Management (basic limits, circuit breakers)

---

### ## 10. Future Enhancements (Phase 2+)

- Swing/Position/Scalping trading
- Leaderboards, community, chat
- AI/ML prediction assistant
- Combo packs, influencer portfolios
- Advanced analytics, liquidity tools

---

### ## 11. Appendix: Key Flows

#### ### 11.1. Trade Flow

1. User places order (buy/sell/short)
2. Trading Service validates, locks funds/margin

3. Order Book matches, executes trade
4. Portfolio/Wallet updated, notification sent
5. Pricing Engine recalculates price

#### #### 11.2. Live Data Flow

1. Match Data Service ingests live feed
2. Player stats updated in DB
3. Pricing Engine recalculates prices
4. Real-time feed pushed to clients

---

### ## 12. CricketIQ Liquidity Engine & Synthetic Player Trading

#### #### 12.1. Tokenized Players (Synthetic Assets)

##### ##### Concept & Rationale

- Each cricket player is represented as a synthetic asset, also referred to as a player token (e.g., PLAYER\_X token).
- These tokens are not real blockchain assets but are managed within the platform's internal ledger for regulatory compliance, speed, and user experience.
- The synthetic asset model allows for rapid price updates, instant settlement, and flexible integration with performance data.
- Tokenization enables fractional ownership, so users can buy any amount (e.g., 0.01 of a player token), increasing accessibility and liquidity.
- The system is designed to mimic the behavior of real-world stocks, making it intuitive for users familiar with financial markets.

##### ##### Token Lifecycle

- **Creation:** Tokens are created for each player in the match/tournament roster. New tokens can be launched for debutants or special events (e.g., Player IPOs).
- **Activation:** Tokens become tradable when a match or tournament starts. Inactive tokens (injured, benched players) are frozen or delisted.
- **Trading:** Users can buy/sell tokens during live matches, with prices updating in real time based on demand and player performance.

- **\*\*Settlement:\*\*** At the end of a match or tournament, open positions are settled based on final player stats and closing price.
- **\*\*Retirement:\*\*** Tokens for retired or permanently delisted players are archived, and any remaining positions are settled at a fair value.

#### ##### Token Properties

- **\*\*Token ID:\*\*** Unique identifier (UUID or slug, e.g., PLAYER\_VIRAT\_KOHLI\_2026).
- **\*\*Player Metadata:\*\*** Name, team, role, photo, bio, and historical stats.
- **\*\*Current Price:\*\*** The latest market price, updated every few seconds.
- **\*\*Circulating Supply:\*\*** Total number of tokens in user portfolios and liquidity pools.
- **\*\*Performance Index:\*\*** Real-time multiplier based on live stats (runs, wickets, milestones, etc.).
- **\*\*Status:\*\*** Active, frozen, delisted, or archived.

#### ##### Price Calculation

- The price of each token is determined by a combination of market demand (order book or AMM) and a performance-driven multiplier.
- Example formula:  $\text{Token Price} = \text{Base Price (from trading)} \times \text{Performance Index (0.8 - 1.2)}$ .
- The performance index is recalculated every few seconds using live data feeds.
- Price history is stored for charting, analytics, and auditing.

#### ##### Token Metadata API

- Exposes endpoints for fetching token details, price history, and player stats.
- Example: `GET /api/tokens/{player_id}` returns metadata, current price, and performance index.
- Used by mobile/web clients to display live token cards, charts, and overlays.

#### ##### Edge Cases & Safeguards

- **\*\*Injury/Withdrawal:\*\*** If a player is injured mid-match, trading may be paused and the token price frozen to prevent manipulation.
- **\*\*Delisting:\*\*** If a player is banned or removed, the token is delisted and positions are settled at a fair value.
- **\*\*Abandoned Matches:\*\*** If a match is abandoned, all open positions are refunded or settled at the pre-abandonment price.

- **Suspicious Activity:** Automated monitoring for pump/dump or wash trading; circuit breakers can freeze trading on abnormal activity.

#### #### User Experience

- Users can search, filter, and favorite player tokens.
- Each token has a detailed profile page with live price, stats, news, and trading options.
- Fractional trading allows users to participate with any budget.
- Leaderboards show top-performing tokens and traders.

#### #### Technical Implementation

- Tokens are stored in the database with all metadata, price history, and status.
- Real-time price updates are pushed to clients via WebSocket channels.
- All trades and price changes are auditable for compliance and dispute resolution.
- The system supports batch creation and management of tokens for large tournaments.

#### #### Integration Points

- **Match Data Service:** Supplies live player stats for performance index calculation.
- **Trading Service:** Manages order book/AMM and executes trades.
- **Portfolio Service:** Tracks user holdings and PnL for each token.
- **Notification Service:** Alerts users to major price moves, milestones, or token events.

#### #### Security & Compliance

- All token transactions are logged and auditable.
- KYC/AML checks are enforced for large trades or withdrawals.
- User data and token holdings are encrypted at rest and in transit.
- Regulatory compliance is maintained by keeping tokens synthetic (not real crypto assets).

#### #### Business Logic

- The company can launch special tokens (e.g., All-Star tokens, combo packs) for marketing and engagement.
- Token supply can be adjusted for special events (e.g., limited edition tokens).
- Fees are charged on each trade, contributing to company revenue and liquidity pool reserves.

#### #### Analytics & Reporting

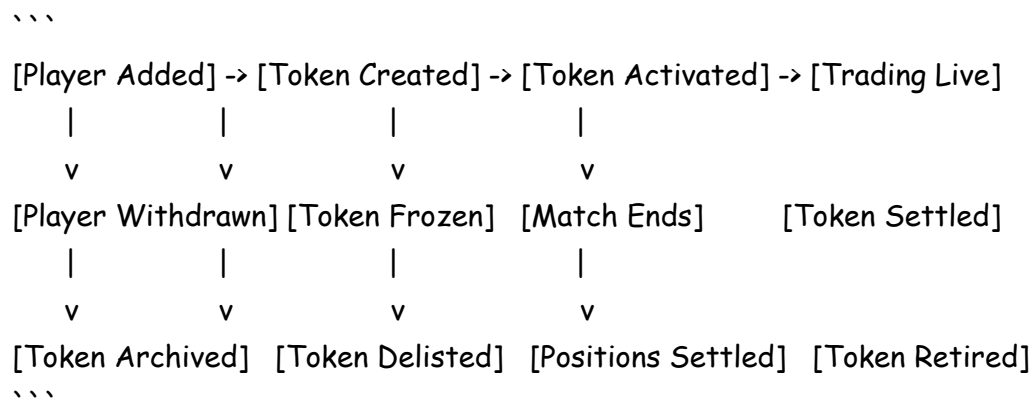
- Real-time dashboards show token trading volume, price volatility, and user participation.

- Historical data is available for backtesting strategies and regulatory reporting.
- Insights into most traded tokens, top gainers/losers, and user trading patterns.

#### #### Example Token Data Structure

```
```json
{
  "token_id": "PLAYER_VIRAT_KOHLI_2026",
  "player_name": "Virat Kohli",
  "team": "India",
  "role": "Batter",
  "status": "active",
  "current_price": 102.5,
  "performance_index": 1.12,
  "circulating_supply": 100000,
  "price_history": [ ... ],
  "metadata": { ... }
}
```
```

#### #### Diagram: Token Lifecycle



#### #### Future Extensions

- Support for multi-match tokens (e.g., series or season-long tokens).
- Integration with external wallets or blockchain for real asset issuance (if regulations allow).
- Gamification: badges, achievements, and rewards for trading activity.



- Social features: share token picks, follow top traders, and community voting on new tokens.

## ### 12.2. Trading Mechanisms (Order Book & AMM)

### #### Overview

- The platform supports two primary trading mechanisms for synthetic player tokens: the Order Book model and the Automated Market Maker (AMM) model.
- Both models are designed to provide liquidity, enable price discovery, and facilitate seamless trading for users, but they differ in execution logic, user experience, and risk profiles.

### #### Order Book Model

- The order book is a traditional exchange mechanism where all buy (bid) and sell (ask) orders are listed at various price levels.
- Users can place market orders (execute immediately at best available price) or limit orders (execute only at a specified price or better).
- The order book displays market depth, showing the quantity of tokens available at each price point, providing transparency and enabling advanced trading strategies.
- Trades are executed when a buy order matches a sell order at a mutually agreed price; this transaction sets the new market price for the token.
- Market makers (company or users) can provide liquidity by continuously placing buy and sell orders, narrowing the bid-ask spread and improving execution speed.
- The system supports partial fills, where an order can be partially executed if there is not enough liquidity at the desired price.
- Order matching is handled by a high-performance matching engine, ensuring fairness (FIFO or pro-rata logic) and low latency.
- All open orders are stored in the database and can be cancelled or modified by users until filled or expired.
- The order book model allows for advanced order types, such as stop-loss, take-profit, and iceberg orders, catering to both casual and professional traders.
- Circuit breakers and price bands are implemented to prevent flash crashes and excessive volatility.

### #### AMM (Automated Market Maker) Model

- The AMM model is a decentralized trading mechanism where trades are executed against a liquidity pool, not directly with another user.
- Liquidity providers (LPs) deposit pairs of assets (e.g., PLAYER\_X token and USDT) into the pool, earning a share of trading fees in return.
- The most common AMM formula is the constant product ( $x * y = k$ ), which ensures that the product of the two asset reserves remains constant after each trade.
- Prices are determined algorithmically based on the ratio of assets in the pool; large trades can cause slippage, especially in low-liquidity pools.
- Anyone can become a liquidity provider, democratizing market making and increasing overall liquidity.
- LPs face the risk of impermanent loss if the price of the token changes significantly relative to the paired asset.
- AMMs provide always-on liquidity, ensuring that users can trade at any time, even if there are no matching counterparties.
- The platform can support multiple pools per token (e.g., PLAYER\_X/USDT, PLAYER\_X/ETH), allowing for cross-asset trading and arbitrage opportunities.
- Dynamic fee structures can be implemented to incentivize liquidity during high-volatility periods or for less popular tokens.
- AMM pools are monitored for health, and rebalancing mechanisms can be triggered if reserves fall below safe thresholds.

#### #### Hybrid Model & Flexibility

- The platform can offer both models, allowing users to choose their preferred trading experience or switch based on market conditions.
- A hybrid approach can route orders to the order book or AMM based on liquidity, price, and user preference, optimizing execution quality.
- Internal arbitrage bots can help keep prices aligned between the order book and AMM pools, reducing fragmentation and improving efficiency.

#### #### Technical Implementation

- The order book and AMM engines are implemented as separate microservices, each with dedicated APIs, databases, and scaling policies.
- Real-time updates are pushed to clients via WebSocket channels, ensuring users see the latest prices, order book depth, and pool balances.
- All trades, order placements, and liquidity events are logged for auditing, analytics, and dispute resolution.

- The system supports high throughput and low latency, leveraging in-memory data stores (e.g., Redis) and efficient matching algorithms.

#### #### User Experience

- Users can view live order books, place advanced orders, and monitor their open positions in real time.
- AMM interfaces display pool sizes, expected slippage, and LP rewards, helping users make informed decisions.
- Educational overlays explain the differences between models, risks, and optimal strategies for new users.

#### #### Risk Management & Safeguards

- Circuit breakers, price bands, and slippage limits protect users from extreme market moves and manipulation.
- Automated monitoring detects wash trading, front-running, and other abusive behaviors, triggering alerts or halting trading if necessary.
- LPs can withdraw liquidity at any time, but withdrawal fees or cooldowns may apply during periods of high volatility.

#### #### Business Logic & Revenue

- The company earns fees on every trade (maker/taker in order book, swap fee in AMM), contributing to revenue and reserve pools.
- Fee structures can be adjusted dynamically to balance liquidity, incentivize participation, and manage risk.
- Special events (e.g., double fee days, liquidity mining campaigns) can be launched to boost engagement and liquidity.

#### #### Analytics & Reporting

- Real-time dashboards show trading volume, liquidity depth, spread analysis, and user participation for both models.
- Historical data enables backtesting, strategy development, and regulatory reporting.
- Insights into most traded tokens, top LPs, and order book/AMM health are available to admins and users.

#### #### Example Order Book Data Structure

```json

```
{
  "bids": [ { "price": 101.5, "quantity": 200 }, ... ],
  "asks": [ { "price": 102.0, "quantity": 150 }, ... ],
  "last_trade": { "price": 101.8, "quantity": 50, "timestamp": "..." }
}
...`
```

Example AMM Pool Data Structure

```
```json
{
 "token_a": "PLAYER_X",
 "token_b": "USDT",
 "reserve_a": 50000,
 "reserve_b": 100000,
 "total_lp_tokens": 1000,
 "fee_rate": 0.003
}
...`
```

#### ##### Diagram: Trading Mechanisms

```
...
[User Places Order] -> [Order Book Engine] -> [Order Matched?] --Yes--> [Trade
Executed]
 |
 No
 |
[Route to AMM Pool] -> [Trade Executed]
...`
```

#### ##### Future Extensions

- Support for additional order types (e.g., trailing stop, OCO, TWAP/VWAP).
- Integration with external DEXs or CEXs for cross-platform liquidity.
- On-chain AMM pools if regulations allow, enabling true DeFi trading.
- Gamification: trading competitions, LP leaderboards, and rewards for active participation.
- Social trading features: copy top traders, share order book strategies, and community-driven liquidity pools.

### ### 12.3. Performance-Driven Price Adjustments

#### #### Overview

- Performance-driven price adjustments are at the heart of the CricketIQ platform, ensuring that player token prices reflect both market sentiment and real-world cricket performance.
- This mechanism creates a dynamic, skill-based trading environment where users are rewarded for accurately predicting player form and match events.

#### #### Data Sources & Integration

- The platform integrates with multiple real-time sports data APIs (e.g., SportRadar, CricAPI, Roanuz) to ingest ball-by-ball stats, player milestones, and match context.
- Data is normalized and validated by the Match Data Service before being used in price calculations.
- Redundant feeds and fallback providers ensure data reliability and minimize downtime during live events.

#### #### Performance Metrics

- Key metrics include runs, wickets, strike rate, economy rate, catches, milestones (e.g., 50s, 100s, 5-wicket hauls), and negative events (ducks, dropped catches, poor economy).
- Contextual factors such as match stage (powerplay, death overs), pitch conditions, and opposition strength can be incorporated for advanced modeling.
- Each metric is assigned a weight based on its impact on match outcome and fantasy value.

#### #### Price Adjustment Formula

- The core formula:  $\text{Token Price} = \text{Base Price (from trading)} \times \text{Performance Index (0.8 - 1.2)}$ .
- The Performance Index is a composite score calculated from live stats, normalized to prevent extreme swings.
- Example: A player scoring a century and taking wickets in a match may see their index rise to 1.18, boosting token price by 18% over the base.
- Negative events (e.g., golden duck, expensive bowling) reduce the index, causing price drops.

#### #### Real-Time Updates

- The Pricing Engine recalculates the Performance Index every 3-5 seconds (or faster for scalping modes).
- Price updates are pushed to all clients via WebSocket, ensuring traders see the latest values instantly.
- All adjustments are logged for transparency, analytics, and dispute resolution.

#### #### Normalization & Fairness

- Performance is normalized across different match types (T20, ODI, Test) to ensure fair comparison and prevent format bias.
- Historical averages and rolling windows are used to smooth out anomalies and reward consistent performers.
- Leaderboards highlight both short-term form (last 5 matches) and long-term consistency (seasonal averages).

#### #### Safeguards & Anti-Manipulation

- Maximum and minimum multipliers are capped per match to prevent manipulation or flash crashes.
- Circuit breakers can freeze price adjustments during data anomalies, suspicious activity, or API outages.
- Manual overrides are available for admins in case of exceptional events (e.g., match fixing, data errors).

#### #### User Experience

- Users see live price charts with performance overlays, showing how stats impact token value in real time.
- Tooltips and analytics explain the impact of each event (e.g., "+5% for half-century").
- Notifications alert users to major price moves, milestones, or unexpected drops.

#### #### Technical Implementation

- The Pricing Engine is a stateless microservice, consuming data from the Match Data Service and Trading Service.
- All calculations are performed in-memory for speed, with results persisted to the database for audit and analytics.
- The system supports backtesting and simulation for new pricing models and parameter tuning.

#### ##### Business Logic

- The company can adjust metric weights, normalization windows, and multiplier caps to fine-tune market behavior.
- Special events (e.g., double points for finals) can temporarily boost performance multipliers for marketing and engagement.
- Performance-driven adjustments increase trading volume and user engagement, driving platform revenue.

#### ##### Analytics & Reporting

- Real-time dashboards show the impact of performance events on token prices, trading volume, and user PnL.
- Historical data enables analysis of model accuracy, user prediction skill, and market efficiency.
- Insights into most sensitive metrics, top-performing players, and volatility drivers are available to admins.

#### ##### Example Performance Index Calculation

```
```json
{
  "runs": 75,
  "wickets": 2,
  "strike_rate": 145.0,
  "milestones": ["50 runs"],
  "negative_events": ["dropped catch"],
  "context": "Powerplay",
  "performance_index": 1.14
}
```
```

#### ##### Diagram: Price Adjustment Flow

...

[Live Match Data] -> [Match Data Service] -> [Pricing Engine] -> [Performance Index Calculation]

|

v

|

v

[Trading Service] <----- [Price Update]

...

#### ##### Future Extensions

- Incorporate advanced metrics (e.g., win probability added, player impact scores).
- Machine learning models to predict future performance and adjust prices proactively.
- User-driven prediction markets for additional price signals.
- Integration with video highlights and news for sentiment-based adjustments.
- Customizable performance indices for user-created tokens or fantasy leagues.

### ### 12.4. Synthetic Stocks: Player Token Logic

#### ##### Overview

- Synthetic stocks in the CricketIQ platform are digital representations of real cricket players, designed to behave like financial stocks but driven by both market speculation and real-world performance.
- Each player token is a unique, tradable asset with its own price, supply, and performance history, enabling users to invest, speculate, and manage portfolios as they would in traditional equity markets.

#### ##### Token Fundamentals

- **\*\*Underlying Asset:\*\*** Each token is mapped to a real player, with metadata including name, team, role, and season.
- **\*\*Synthetic Nature:\*\*** Tokens are not issued on a public blockchain but are managed within the platform's secure, centralized ledger for compliance and speed.
- **\*\*Fractional Ownership:\*\*** Users can buy and sell fractions of tokens, increasing accessibility and liquidity for all participants.
- **\*\*Supply Model:\*\*** The circulating supply of each token is determined by initial issuance (e.g., player IPO), subsequent trading, and company or community-driven events (e.g., burns, splits).

#### ##### Price Drivers

- **\*\*Market Demand:\*\*** Buy and sell activity in the order book or AMM pools directly impacts token price, reflecting trader sentiment and speculation.
- **\*\*Performance Index:\*\*** Real-time player stats act as a multiplier, ensuring prices reflect both hype and actual on-field results.



- **External Events:** News, injuries, transfers, and off-field events can trigger price adjustments, either automatically (via oracles) or manually (admin intervention).

#### #### Trading & Portfolio Management

- Users can buy, sell, or short player tokens, building diversified portfolios based on matchups, form, and personal strategy.
- Portfolio dashboards show real-time PnL, historical trades, and sector breakdowns (e.g., batters, bowlers).
- Advanced order types (limit, stop-loss, take-profit) are supported for risk management.

#### #### Token Lifecycle Events

- **IPO/Launch:** New tokens are created for debutants or special events, with initial pricing set by auction, bonding curve, or admin decision.
- **Splits & Merges:** Tokens can be split (increase supply, lower price) or merged (reduce supply, increase price) to manage liquidity and user experience.
- **Burns:** Tokens can be burned (removed from circulation) as part of promotions, penalties, or to manage inflation.
- **Delisting:** Tokens for retired, banned, or inactive players are delisted, with positions settled at a fair value.

#### #### Technical Architecture

- Tokens are stored as rows in the database, with fields for player ID, supply, price, status, and metadata.
- All trades, transfers, and lifecycle events are logged for auditability and compliance.
- The system supports batch operations for large tournaments (e.g., World Cup), enabling efficient token management.
- APIs expose token data, price history, and event logs for integration with mobile/web clients and analytics tools.

#### #### Security & Compliance

- All token transactions are subject to KYC/AML checks, especially for large trades or withdrawals.
- Synthetic tokens are not securities or real crypto assets, reducing regulatory risk and simplifying onboarding.
- User holdings and transaction data are encrypted at rest and in transit.

- Admin controls allow for emergency intervention in case of bugs, exploits, or regulatory requests.

#### ##### Business Logic

- The company can issue special tokens (e.g., All-Star, combo packs) for marketing, engagement, and monetization.
- Fees are charged on every trade, with a portion allocated to liquidity pools, insurance funds, and company revenue.
- Token buybacks, burns, and airdrops can be used to manage supply, reward users, and drive engagement.

#### ##### User Experience

- Each token has a detailed profile page with live price, stats, news, and trading options.
- Users can set price alerts, follow favorite tokens, and share picks with friends or on social media.
- Leaderboards and achievements gamify trading, rewarding top performers and active participants.

#### ##### Edge Cases & Safeguards

- **\*\*Suspicious Activity:\*\*** Automated monitoring for wash trading, pump/dump, and other manipulative behaviors.
- **\*\*Data Anomalies:\*\*** Circuit breakers and manual overrides for price errors or data feed issues.
- **\*\*Inactive Players:\*\*** Tokens for players not in the squad are frozen or delisted to prevent unfair trading.

#### ##### Analytics & Reporting

- Real-time dashboards show token trading volume, volatility, and user participation.
- Historical data enables backtesting, strategy development, and regulatory reporting.
- Insights into top LPs, most profitable pools, and auction outcomes inform future strategy.

#### ##### Example Token Data Structure

```
```json
{
  "token_id": "PLAYER_JOS_BUTTLER_2026",
```

```

"player_name": "Jos Buttler",
"team": "England",
"role": "Wicketkeeper-Batter",
"status": "active",
"current_price": 98.7,
"circulating_supply": 75000,
"performance_index": 1.09,
"price_history": [ ... ],
"metadata": { ... }
}
...

```

Diagram: Token Logic Flow

...

[Player Event] -> [Token Created] -> [Trading Live] -> [Performance Update] -> [Price Adjusted]

```

      |           |           |           |           |
      v           v           v           v           v

```

[IPO/Auction] [User Buys/Sells] [Stats Change] [Order Book/AMM] [Portfolio Updated]

...

Future Extensions

- On-chain token issuance for regulatory-compliant markets.
- User-created tokens for fantasy leagues or custom competitions.
- Integration with NFT collectibles (e.g., player cards, match moments).
- Social trading features: copy portfolios, influencer tokens, and community-driven events.

12.5. Company Profit/Loss & Money Flow

Overview

- The CricketIQ platform is designed to be a sustainable business, generating revenue through trading activity, liquidity provision, and special events, while managing risk and ensuring fair play for all participants.
- Understanding the flow of money and profit/loss dynamics is critical for both company strategy and user trust.

Revenue Streams

- **Trading Fees:** The company charges a fee on every trade (maker/taker in order book, swap fee in AMM), typically a small percentage of the transaction value.
- **Liquidity Pool Spreads:** If the company acts as a liquidity provider, it earns the spread between buy and sell prices in the pool.
- **Token Issuance:** Initial player token launches (IPOs) can generate upfront revenue as users buy into new assets.
- **Special Events:** Revenue from tournaments, fantasy contests, and premium features (e.g., advanced analytics, leaderboards).
- **Withdrawal/Deposit Fees:** Optional fees on fiat or crypto deposits/withdrawals to cover payment processing costs.
- **Advertising & Sponsorships:** Monetization through branded tournaments, in-app ads, and partnerships.

Cost Centers

- **Infrastructure:** Cloud hosting, data feeds, and real-time processing costs.
- **Liquidity Incentives:** Rewards paid to user liquidity providers (LPs) to bootstrap and maintain healthy pools.
- **Insurance/Reserve Pools:** Funds set aside to cover extreme market events, user protection, and regulatory requirements.
- **Compliance & Security:** KYC/AML checks, audits, and cybersecurity measures.
- **Customer Support:** Staff and tooling for user support, dispute resolution, and community management.

Money Flow Diagram

\\`

[User Deposits] ----> [Platform Wallet] ----> [Trading Engine]

| | |
v v v

[Liquidity Pools] <--- [Company Treasury] <--- [Trading Fees]

| | |
v v v

[LP Rewards] [Insurance Pool] [Operational Costs]

| | |
v v v

[User Withdrawals] <--- [Reserve Fund] <--- [Profit/Loss]
...

Trading Flow

- Users deposit funds (fiat, crypto, or platform credits) into their wallets.
- Funds are used to buy/sell player tokens, with each trade incurring a fee.
- Fees are split between company revenue, LP rewards, and insurance pools.
- Profitable traders can withdraw gains, while losses are absorbed by their own balances or, in rare cases, the insurance pool.

Liquidity Provision

- The company may seed initial liquidity in order book or AMM pools, but the goal is to transition to user-provided liquidity over time.
- LPs earn a share of trading fees proportional to their contribution, incentivizing participation and deepening markets.
- Dynamic fee structures and bonus rewards can be used to attract liquidity during high-demand periods or for new tokens.

Risk Management

- The company manages risk through circuit breakers, margin requirements, and real-time monitoring of pool health.
- Insurance and reserve pools are funded from trading fees, providing a safety net for extreme events (e.g., flash crashes, data errors).
- Exposure limits and cooldowns prevent runaway losses and protect both users and the company.

Profit/Loss Calculation

- **Gross Revenue:** Sum of all trading fees, spreads, and event income.
- **Net Profit:** Gross revenue minus infrastructure, incentives, insurance, and operational costs.
- **LP Payouts:** Calculated based on pool share and trading volume, paid out periodically or on demand.
- **Reserve Growth:** A portion of profits is allocated to the reserve fund for long-term sustainability.

Transparency & Reporting

- Real-time dashboards show company revenue, trading volume, LP rewards, and reserve balances.
- Regular financial reports are published for user trust and regulatory compliance.
- All fee structures, payout formulas, and reserve policies are documented and auditable.

User Perspective

- Users see transparent fee breakdowns on every trade and withdrawal.
- LP dashboards show real-time earnings, pool health, and historical returns.
- Leaderboards and analytics highlight top traders, LPs, and most profitable tokens.

Company Perspective

- The company monitors key metrics: daily active traders, gross volume traded, fee revenue, LP participation, and reserve adequacy.
- Scenario analysis and stress testing are performed to ensure resilience under extreme market conditions.
- Strategic decisions (e.g., fee changes, new token launches) are data-driven and aligned with long-term growth.

Edge Cases & Safeguards

- ****Liquidity Crunch:**** Emergency protocols to inject company funds or pause trading if pools become illiquid.
- ****Market Manipulation:**** Automated detection and manual review of suspicious trading patterns.
- ****Regulatory Action:**** Reserve funds and compliance teams ready to respond to audits, fines, or legal changes.

Future Extensions

- Dynamic fee models based on market volatility, token popularity, or user tier.
- Integration with DeFi protocols for external yield generation on reserve funds.
- Community governance over fee structures, reserve allocation, and new token launches.
- Expansion into new sports, asset classes, or cross-platform trading partnerships.

12.6. Bootstrapping Liquidity (Without Company Funding)

Overview

- Bootstrapping liquidity is essential for a sustainable trading platform, especially when the company aims to minimize or avoid direct capital injection.
- The goal is to create a self-sustaining ecosystem where users, not the company, provide the liquidity needed for active trading and price discovery.

A. Player-Provided Liquidity (Skin in the Game)

- Users deposit their own funds (fiat, crypto, or platform credits) to participate in trading and liquidity provision.
- These deposits form the liquidity pool, enabling buy/sell activity and reducing reliance on company capital.
- The company acts as a coordinator and fee-taker, not a liquidity provider.
- Incentives for users include a share of trading fees, LP rewards, and potential bonus tokens for early participation.
- Risks for users are mitigated by transparent pool health metrics, withdrawal options, and insurance funds.
- Governance mechanisms can allow LPs to vote on pool parameters, fee rates, and new token launches.

B. Synthetic Liquidity (Virtual Balances)

- The platform can issue synthetic credits to users at signup (e.g., 1,000 virtual credits each).
- Trading occurs in a simulated market, with price discovery driven by user activity and performance data.
- No real money is at risk initially, making it ideal for MVPs, user onboarding, and stress testing.
- Once the system stabilizes and user demand is proven, conversion events allow users to cash out or transition to real liquidity.
- Synthetic liquidity helps build a user base, gather data, and refine trading models before introducing real capital.
- Drawbacks include limited trust and the need for eventual real liquidity to maintain credibility.

C. Peer-to-Peer Settlement

- Trades are settled directly between users, with winners paid from losers' balances.
- The company only charges a small commission on each trade, never taking on market risk.
- This model is fully self-sustaining, as the platform acts as a neutral coordinator.

- Requires a balanced inflow and outflow of funds; if all users win, the company may need to cover shortfalls from reserves.
- Peer-to-peer settlement is transparent, auditable, and aligns incentives for fair play.

D. NFT / Token Collateralization

- Users can stake NFTs or platform tokens as collateral to unlock trading power and liquidity.
- Collateralized assets secure the system, allowing for leveraged trading and risk management.
- Staked assets can be slashed or locked in case of rule violations or excessive losses.
- This approach drives demand for platform tokens and decentralizes liquidity provision.
- Complex UX and trust in system rules are challenges to adoption.

E. Auction-Style Liquidity Matching

- Instead of fixed pools, trades are matched via periodic auctions where users submit bids and asks.
- Trades only clear when counterparties exist, ensuring no company funds are needed.
- Auctions can be time-based (e.g., every 5 minutes) or event-driven (e.g., after each over).
- This model may feel less liquid but is highly capital efficient and transparent.

Dynamic Incentives & Safeguards

- Dynamic fee structures can reward early LPs, high-volume traders, or those providing liquidity to less popular tokens.
- Insurance pools, funded from trading fees, act as a backstop for extreme events or imbalances.
- Burn/recycle mechanisms can reduce token supply, increase scarcity, and reward long-term participants.
- Withdrawal cooldowns and minimum lock periods prevent liquidity runs and stabilize pools.

Technical Implementation

- Smart contracts or platform logic manage deposits, withdrawals, LP shares, and fee distribution.
- Real-time dashboards show pool health, user contributions, and expected rewards.
- All transactions are logged for auditability and regulatory compliance.
- APIs expose pool data, LP stats, and auction results for integration with mobile/web clients.

User Experience

- Onboarding flows explain liquidity provision, risks, and rewards.
- Users can track their LP shares, earnings, and withdrawal eligibility in real time.
- Educational content and gamification (e.g., LP leaderboards, badges) drive engagement.

Business Logic

- The company can adjust incentive structures, pool parameters, and auction frequency based on market conditions.
- Community governance can be introduced for key decisions, increasing user ownership and loyalty.
- Special events (e.g., liquidity mining campaigns) can boost participation during launches or high-demand periods.

Analytics & Reporting

- Real-time analytics track liquidity depth, pool utilization, LP participation, and fee distribution.
- Historical data enables analysis of incentive effectiveness, user retention, and market resilience.
- Insights into top LPs, most profitable pools, and auction outcomes inform future strategy.

Edge Cases & Safeguards

- ****Liquidity Crunch:**** Emergency protocols to pause trading, increase fees, or trigger insurance payouts.
- ****Sybil Attacks:**** KYC/AML checks and anti-fraud measures prevent abuse of synthetic or real liquidity.
- ****Regulatory Changes:**** Flexible architecture allows for rapid adaptation to new rules or market conditions.

Future Extensions

- Integration with external DeFi protocols for additional yield and liquidity.
- Cross-platform liquidity sharing with partner exchanges or fantasy platforms.
- User-created pools, custom tokens, and community-driven liquidity events.
- Advanced risk management tools: dynamic margin, volatility-adjusted rewards, and automated rebalancing.

12.8. Sustainability & Safeguards

Overview

- Sustainability and robust safeguards are critical for the long-term viability of the CricketIQ platform, ensuring user trust, regulatory compliance, and operational resilience.
- The system is designed to withstand market shocks, manipulation attempts, and evolving business or regulatory requirements.

Dynamic Fee Structures

- Trading fees are dynamically adjusted based on market volatility, liquidity depth, and token popularity.
- Higher fees during periods of extreme volatility help dampen speculation and build reserve funds.
- Fee discounts or rebates can incentivize liquidity provision, long-term holding, or participation in special events.

Insurance & Reserve Pools

- A portion of all trading fees is allocated to insurance and reserve pools, which act as a safety net for users and the company.
- These pools cover losses from extreme events (e.g., flash crashes, data feed errors, smart contract bugs).
- Reserve pool balances and payout policies are transparent and auditable, with regular reporting to users and regulators.

Circuit Breakers & Price Bands

- Automated circuit breakers halt trading or freeze token prices during abnormal market moves, data anomalies, or suspected manipulation.
- Price bands limit the maximum allowed price movement per interval (e.g., $\pm 20\%$ per hour), reducing the risk of flash crashes.
- Manual intervention by admins is possible in case of emergencies, with all actions logged and reviewed.

Anti-Manipulation & Abuse Detection

- Real-time monitoring detects suspicious trading patterns, wash trading, front-running, and pump/dump schemes.

- Automated alerts and rule-based actions (e.g., order cancellation, account suspension) protect market integrity.
- KYC/AML checks and transaction limits prevent Sybil attacks and money laundering.

User Protection Mechanisms

- Margin requirements and position limits prevent users from taking on excessive risk or leveraging beyond their means.
- Volatility warnings, margin call alerts, and educational content help users make informed decisions.
- Dispute resolution processes and customer support are in place for handling user complaints or technical issues.

Platform Health Monitoring

- Real-time dashboards track liquidity, trading volume, pool utilization, and system performance.
- Automated health checks and failover mechanisms ensure high availability and rapid recovery from outages.
- Regular stress testing and scenario analysis prepare the platform for extreme market conditions.

Regulatory Compliance

- The platform adheres to all relevant financial, data privacy, and consumer protection regulations.
- Regular audits, compliance reviews, and legal updates ensure ongoing alignment with global standards.
- User data is encrypted, access-controlled, and subject to GDPR/CCPA requirements.

Community Governance & Transparency

- Users and liquidity providers can participate in governance, voting on key parameters (e.g., fee rates, pool allocation).
- All major changes to platform rules, fee structures, or token listings are announced in advance and open to community feedback.
- Transparency reports detail reserve balances, insurance payouts, and incident responses.

Burn/Recycle Mechanisms

- A portion of trading fees or token supply can be burned (permanently removed) to reduce inflation and increase scarcity.
- Recycle mechanisms redirect fees to user rewards, liquidity incentives, or community grants.
- Burn events are announced and verifiable, building trust and long-term value.

Fail-Safes & Emergency Protocols

- Emergency pause functions allow for rapid shutdown of trading or withdrawals in case of critical bugs or attacks.
- Multi-signature admin controls and access logs prevent unauthorized actions.
- Incident response plans and communication protocols ensure users are informed and protected during crises.

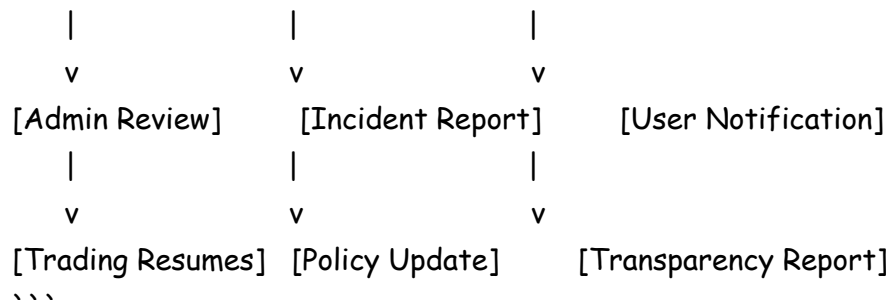
Future-Proofing

- Modular architecture allows for rapid adaptation to new regulations, market trends, or user needs.
- Continuous improvement through user feedback, analytics, and industry best practices.
- Expansion into new asset classes, sports, or geographies is supported by flexible system design.

Example Safeguard Flow

...

[Abnormal Price Move] -> [Circuit Breaker Triggered] -> [Trading Halted]



Analytics & Reporting

- Real-time and historical analytics track the effectiveness of safeguards, user protection, and platform health.
- Regular reports are published to users, LPs, and regulators, building trust and accountability.

Edge Cases & Lessons Learned

- Post-mortems are conducted after major incidents, with findings shared transparently.
- Lessons learned inform future safeguard enhancements and risk management strategies.

``````markdown

## ## 6. WebSocket API Gateway Endpoints (Real-Time Data)

#### #### Overview

- The API Gateway exposes WebSocket endpoints for real-time data delivery to web and mobile clients.
- WebSocket is chosen for its bidirectional, low-latency communication, ideal for live trading, price feeds, and notifications.
- All endpoints are authenticated via JWT and support subscription/unsubscription to specific data channels.
- Payloads are JSON-encoded for interoperability and ease of use.

#### #### Connection & Authentication

- **Endpoint:** `/ws/connect`

- **Request:**

``json

{

  "token": "<JWT access token>"

}

``

- **Response:**

``json

{

  "status": "connected",

  "user\_id": "<uuid>",

  "channels": [ ]

}

``

- On connection, the server validates the JWT and establishes a session.

#### #### Subscribe to Player Token Price Feed

- **Endpoint:** `/ws/subscribe/price`

- **\*\*Request:\*\***

```
```json
{
  "action": "subscribe",
  "channel": "price",
  "player_token_ids": ["PLAYER_VIRAT_KOHLI_2026",
"PLAYER_JOS_BUTTLER_2026"]
}
```
```

- **\*\*Response (on update):\*\***

```
```json
{
  "channel": "price",
  "data": [
    {
      "token_id": "PLAYER_VIRAT_KOHLI_2026",
      "current_price": 102.5,
      "performance_index": 1.12,
      "timestamp": "2026-01-30T12:34:56Z"
    },
    ...
  ]
}
```
```

#### #### Subscribe to Order Book Updates

- **\*\*Endpoint:\*\*** `/ws/subscribe/orderbook`

- **\*\*Request:\*\***

```
```json
{
  "action": "subscribe",
  "channel": "orderbook",
  "player_token_id": "PLAYER_VIRAT_KOHLI_2026"
}
```
```

- **\*\*Response (on update):\*\***

```
```json
{
  "channel": "orderbook",
  "token_id": "PLAYER_VIRAT_KOHLI_2026",
  "bids": [ { "price": 101.5, "quantity": 200 }, ... ],
  "asks": [ { "price": 102.0, "quantity": 150 }, ... ],
  "last_trade": { "price": 101.8, "quantity": 50, "timestamp": "..." }
}
```
```

### ### Subscribe to Live Match Events

- **Endpoint:** `/ws/subscribe/match``

- **Request:**

```
```json
{
  "action": "subscribe",
  "channel": "match",
  "match_id": "MATCH_2026_01_30_IND_VS_ENG"
}
```
```

- **Response (on event):**

```
```json
{
  "channel": "match",
  "match_id": "MATCH_2026_01_30_IND_VS_ENG",
  "event": {
    "type": "wicket",
    "player": "PLAYER_JASPRIT BUMRAH_2026",
    "over": 12,
    "ball": 3,
    "description": "Bumrah takes wicket!",
    "timestamp": "2026-01-30T12:45:00Z"
  }
}
```
```

### #### Subscribe to Notifications

```
- **Endpoint:** `/ws/subscribe/notifications`
- **Request:**
 ``json
 {
 "action": "subscribe",
 "channel": "notifications"
 }
 ``
- **Response (on notification):**
 ``json
 {
 "channel": "notifications",
 "data": {
 "type": "price_alert",
 "message": "Virat Kohli token up 10% in last 5 minutes!",
 "timestamp": "2026-01-30T12:50:00Z"
 }
 }
 ``
```

### #### Unsubscribe from Channel

```
- **Endpoint:** `/ws/unsubscribe`
- **Request:**
 ``json
 {
 "action": "unsubscribe",
 "channel": "orderbook",
 "player_token_id": "PLAYER_VIRAT_KOHLI_2026"
 }
 ``
- **Response:**
 ``json
 {
 "status": "unsubscribed",
 "channel": "orderbook",
```



```
"player_token_id": "PLAYER_VIRAT_KOHLI_2026"
}
...
```

#### #### Error Handling

- **\*\*Response (on error):\*\***

```
```json
{
  "error": {
    "code": 401,
    "message": "Invalid token or unauthorized"
  }
}
...`
```

Notes

- All endpoints support both subscribe and unsubscribe actions.
 - Payloads are extensible for future channels (e.g., chat, leaderboard, custom analytics).
 - For ultra-low-latency or peer-to-peer features (e.g., live video, voice chat), WebRTC can be integrated as a separate channel.
 - All messages include timestamps and unique IDs for deduplication and ordering.
-

7. Internal gRPC Endpoints for Microservices

Overview

- Each microservice exposes a set of gRPC endpoints for internal communication, enabling efficient, strongly-typed, and low-latency interactions.
- The API Gateway acts as the orchestrator, receiving client requests (REST/WebSocket), invoking the appropriate gRPC service(s), and aggregating responses before sending them to the user.
- All gRPC messages are defined using Protocol Buffers (protobuf), ensuring schema consistency and code generation across services.

7.1. User Service gRPC Endpoints

- ****AuthenticateUser****

- Request: `{ token: string }`

- Response: `{ user\_id: string, valid: bool, roles: [string] }`
- ****GetUserProfile****
 - Request: `{ user\_id: string }`
 - Response: `{ user: User }`
- ****UpdateProfile****
 - Request: `{ user\_id: string, profile: UserProfileUpdate }`
 - Response: `{ success: bool }`
- ****SubmitKYC****
 - Request: `{ user\_id: string, kyc\_data: KYCData }`
 - Response: `{ status: string }`

7.2. Trading Service gRPC Endpoints

- ****PlaceOrder****
 - Request: `{ user\_id: string, order: OrderRequest }`
 - Response: `{ order\_id: string, status: string }`
- ****CancelOrder****
 - Request: `{ user\_id: string, order\_id: string }`
 - Response: `{ success: bool }`
- ****GetOrderBook****
 - Request: `{ player\_token\_id: string }`
 - Response: `{ bids: [Order], asks: [Order], last\_trade: Trade }`
- ****GetTradeHistory****
 - Request: `{ user\_id: string, player\_token\_id: string }`
 - Response: `{ trades: [Trade] }`

7.3. Pricing Engine gRPC Endpoints

- ****GetCurrentPrice****
 - Request: `{ player\_token\_id: string }`
 - Response: `{ price: float, performance\_index: float, timestamp: string }`
- ****UpdatePerformanceIndex****
 - Request: `{ player\_token\_id: string, stats: PlayerStats }`
 - Response: `{ success: bool }`
- ****GetPriceHistory****
 - Request: `{ player\_token\_id: string, range: TimeRange }`
 - Response: `{ prices: [PricePoint] }`

7.4. Portfolio Service gRPC Endpoints

- **GetPortfolio**
 - Request: `{ user\_id: string }`
 - Response: `{ positions: [PortfolioPosition], pnl: float }`
- **UpdatePosition**
 - Request: `{ user\_id: string, position\_update: PositionUpdate }`
 - Response: `{ success: bool }`
- **GetHistoricalPnL**
 - Request: `{ user\_id: string, range: TimeRange }`
 - Response: `{ pnl\_history: [PnLPoint] }`

7.5. Match Data Service gRPC Endpoints

- **GetLiveMatchData**
 - Request: `{ match\_id: string }`
 - Response: `{ match: Match, player\_stats: [PlayerStats] }`
- **GetPlayerStats**
 - Request: `{ player\_token\_id: string, match\_id: string }`
 - Response: `{ stats: PlayerStats }`
- **GetMatchSchedule**
 - Request: `{ }`
 - Response: `{ matches: [Match] }`

7.6. Notification Service gRPC Endpoints

- **SendNotification**
 - Request: `{ user\_id: string, notification: Notification }`
 - Response: `{ success: bool }`
- **GetUserNotifications**
 - Request: `{ user\_id: string }`
 - Response: `{ notifications: [Notification] }`
- **MarkAsRead**
 - Request: `{ user\_id: string, notification\_id: string }`
 - Response: `{ success: bool }`

7.7. Admin Service gRPC Endpoints

- **GetUserList**
 - Request: `{ filter: UserFilter }`

- Response: `{ users: [User] }`
- ****ApproveKYC****
 - Request: `{ user\_id: string, kyc\_id: string }`
 - Response: `{ status: string }`
- ****GetAuditLogs****
 - Request: `{ range: TimeRange }`
 - Response: `{ logs: [AuditLog] }`

7.8. Community Service gRPC Endpoints

- ****GetLeaderboard****
 - Request: `{ type: string, range: TimeRange }`
 - Response: `{ leaderboard: [LeaderboardEntry] }`
- ****PostChatMessage****
 - Request: `{ user\_id: string, message: ChatMessage }`
 - Response: `{ success: bool }`
- ****GetAchievements****
 - Request: `{ user\_id: string }`
 - Response: `{ achievements: [Achievement] }`

Notes

- All endpoints are versioned and documented via protobuf files (e.g., `user.proto`, `trading.proto`).
- API Gateway handles request validation, authentication, and aggregation of responses before sending to clients via WebSocket.
- gRPC supports streaming for high-frequency data (e.g., price updates, order book changes) if needed.
- Error handling follows a standard schema: `{ code: int, message: string, details?: object }`.